REST APIs

ACM-W
Technical Meeting

Stephanie Bittner
3/9/2026

Contents

- What is an **API?**
- **Example**: Weather
- What is a **REST API**?
- **CRUD Operations in REST APIs**
- Structure of a **URL**
- **Request** and **Response** Components
- Using GET with the **Weather API**
- Using GET/POST/DELETE with the **OpenAI API**
- **Status Code Errors**

What is an API?

- API = **Application Programming Interface**
- A set of rules that allow **different software systems** to **communicate**
- App → API Request → Server → API Response → App

Example: Weather

- When you open a weather app, it **sends a request to a weather API**
- **Request:** GET https://api.weather.com/v1/weather?city=Chicago
- **Response:**

```
{
  "city": “Chicago",
  "temperature": 73.24,
  "condition": “few clouds"
}
```

- The app **displays this data** on the screen.

What is a REST API?

- REST = **Representational State Transfer**
- A **design style** for building web APIs
- Uses standard **HTTP requests**
- Designed to be:
  - Simple
  - Scalable
  - Stateless

CRUD Operations in REST APIs

CRUD describes the four basic actions we perform on data.

CRUD	Action	HTTP Method
Create	Add new data	POST
Read	Get data	GET
Update	Modify existing data	PUT
Delete	Remove data	DELETE

Structure of a URL

Protocol	Subdomain	Domain Name	Top-Level Domain	Endpoint	Query Parameter
https://	api.	weather.	com/	v1/weather	?city=Chicago
How the browser communicates with the server	Shows this is an API service	The main website or service	Type of domain	The specific API resource being requested	Extra information sent with the request

Request Components

Example:

```python
@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json()
    user_message = data.get("message", "")

    response = client.responses.create(
        model="gpt-5.4",
        input=user_message
    )

    return jsonify({
        "id": response.id,
        "reply": response.output_text
    })
```

URL (endpoint): /ask

HTTP Method: POST

Header: Content-Type: application/json

Note: header sent by *data = request.get_json()*

Body (data sent in request; JSON)**:**

{

“message”: “Why did the chicken cross the road?”

}

A request includes a

- **URL** (endpoint)
- **HTTP Method**
- **Headers**
- **Body** (optional)

Response Components

Example:

```python
@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json()
    user_message = data.get("message", "")

    response = client.responses.create(
        model="gpt-5.4",
        input=user_message
    )

    return jsonify({
        "id": response.id,
        "reply": response.output_text
    })
```

A response includes a

- **Status Code**
- **Header(s)**
- **Response Data**

Status Code: 200 OK

Header: Content-Type: application/json

Response Data (JSON)**:**

{

“id”: “resp_12345”

“reply”: “To get to the other side.”

}

Using GET with the Weather API

Process:

- User enters a city name
- JavaScript sends a GET request to the Weather API
- The API returns weather data (JSON)
- The page displays the temperature and description

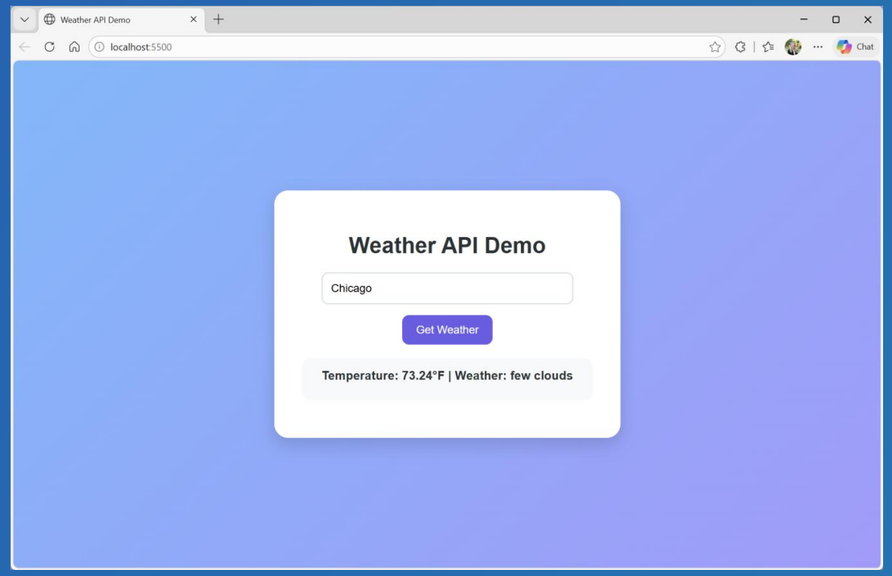
How to get your OpenWeatherMap API Key:

- Create an account at openweathermap.org
- Go to API Keys in your account dashboard
- Copy your generated API key
- Add the key to your code *(here config.js file)*

index.html | script.js | style.css | config.js

script.js > getWeather

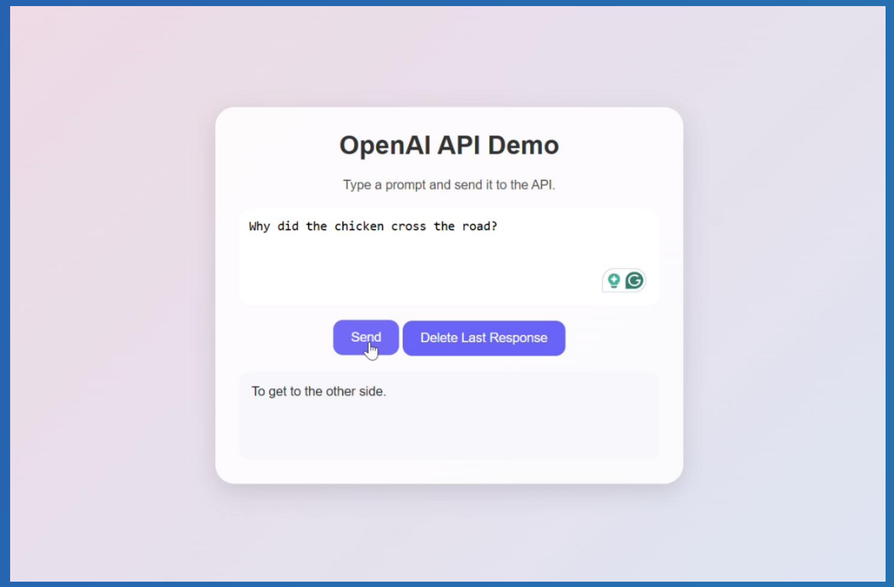
```javascript
const apiKey = CONFIG.API_KEY;
//const apiKey = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

async function getWeather() {

    const city = document.getElementById("cityInput").value;

    const url = `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=imperial`;

    const response = await fetch(url);
    const data = await response.json();

    const temp = data.main.temp;
    const description = data.weather[0].description;

    document.getElementById("result").innerText =
        `Temperature: ${temp}°F | Weather: ${description}`;
}
```



Using GET/POST/DELETE with the OpenAI API

app.py > ...

```python
from flask import Flask, render_template, request, jsonify
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
app = Flask(__name__)
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
# OPENAI_API_KEY=sk-xxxxxxxxxxxx


# ----------------------------------
# GET: show the website homepage
# ----------------------------------
@app.route("/", methods=["GET"])
def home():
    return render_template("index.html")


# ----------------------------------
# POST: create a new OpenAI response
# ----------------------------------
@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json()
    user_message = data.get("message", "")

    response = client.responses.create(
        model="gpt-5.4",
        input=user_message
    )

    return jsonify({
        "id": response.id,
        "reply": response.output_text
    })


# ----------------------------------
# GET: retrieve a previous response by ID
# ----------------------------------
@app.route("/response/<response_id>", methods=["GET"])
def get_response(response_id):
    response = client.responses.retrieve(response_id)

    return jsonify({
        "id": response.id,
        "status": response.status
    })


# ----------------------------------
# DELETE: delete a previous response by ID
# ----------------------------------
@app.route("/response/<response_id>", methods=["DELETE"])
def delete_response(response_id):
    result = client.responses.delete(response_id)

    return jsonify({
        "deleted": True,
        "result": str(result)
    })


if __name__ == "__main__":
    app.run(debug=True)
```




```
(base) ubuntu@StephieBittner:~/ACM-W/openai-api-demo$ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 250-505-669
127.0.0.1 - - [09/Mar/2026 10:30:22] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Mar/2026 10:30:22] "GET /static/script.js HTTP/1.1" 304 -
127.0.0.1 - - [09/Mar/2026 10:30:22] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Mar/2026 10:30:56] "POST /ask HTTP/1.1" 200 -
127.0.0.1 - - [09/Mar/2026 10:31:09] "POST /ask HTTP/1.1" 200 -
127.0.0.1 - - [09/Mar/2026 10:31:11] "DELETE /response/resp_0bc22f261efb23ff0069aee7bdf5f8819590548cc52b4d5886 HTTP/1.1" 200 -
```

Status Code Errors

Every API response includes a **status code**.

It tells us whether the **request worked** or if **something went wrong**.

In our OpenAI API demo, the terminal shows this after each request:

```
"GET / HTTP/1.1" 200 -
"GET /static/script.js HTTP/1.1" 304 -
"GET /static/style.css HTTP/1.1" 304 -
"POST /ask HTTP/1.1" 200 -
"POST /ask HTTP/1.1" 200 -
"DELETE /response/resp_0bc22f261efb23ff0069aee7bdf5f8819590548cc52b4d5886 HTTP/1.1" 200 -
```

Possible Responses:

- **200 OK:** Data was returned successfully
- **304 Not Modified:** Browser already has file, so server does not send it again
- **400 Bad Request:** Request was invalid
- **401 Unauthorized:** API key missing or wrong
- **404 Not Found:** Endpoint or data does not exist
- **500 Internal Server Error:** Server failed

Status Code Categories:

Category	Range
Informational	100-199
Success	200-299
Redirection	300-399
Client Error	400-499
Server Errors	500-599

Thank You